

IA PARA DESENVOLVEDORES [T1]

Situação de Aprendizagem (Mini-Projeto Avaliativo) - Módulo 2 - Semana 05/06

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	2
4. REQUISITOS DA APLICAÇÃO	3
5. ROTEIRO DA APLICAÇÃO	3
5.1. FORMAÇÃO DOS GRUPOS E IDEIA INICIAL	4
5.2. DEFINIÇÃO DO AGENTE	4
5.3. IMPLEMENTAÇÃO COM LANGGRAPH, FERRAMENTA E CONTEXTO	4
5.4. DOCUMENTAÇÃO, PROMPTS E REPOSITÓRIO	5
6. CRITÉRIOS DE AVALIAÇÃO	6
7. CHECKLIST FINAL DE ENTREGA	8

1. CONTEXTUALIZAÇÃO

Este documento descreve o Mini-Projeto Avaliativo do Módulo 2 da disciplina **IA para DEVs**. Nesta etapa, o foco é aplicar os conceitos de agentes de IA em um projeto prático, por meio da construção de uma solução funcional e bem documentada que automatize um processo real com apoio de IA.

O projeto poderá ser desenvolvido individualmente ou em grupos de até 3 alunos. A proposta é criar um agente usando LangGraph, demonstrando o uso de componentes como objetivo, estado, memória, ferramentas, chamadas a APIs ou arquivos, validações e geração de respostas estruturadas.

O agente poderá automatizar tanto processos relacionados ao ciclo de desenvolvimento de software, como revisão de pull requests, análise de logs de pipeline, triagem de issues e apoio à documentação técnica, quanto processos específicos da temática ou case escolhido pelo aluno ou grupo. Em ambos os casos, a solução deve demonstrar um fluxo claro de automação, com entrada definida, processamento por agente, uso controlado de ferramentas e saída útil para o usuário final.

2. DESAFIO

O aluno ou grupo deverá desenvolver um **agente funcional, demonstrável e documentado**, capaz de automatizar um processo real relacionado ao desenvolvimento de software ou à temática escolhida para o projeto. A solução deve apresentar uma entrada definida, um fluxo de execução organizado, uso de ferramenta, tratamento de contexto e uma saída útil para o usuário final.

Ao construir o agente proposto, o aluno estará colocando em prática os aprendizados em:

- Definir um agente com **objetivo claro**, explicando qual processo será automatizado, qual entrada será recebida e qual resultado será entregue ao usuário.
- Implementar um fluxo funcional com **LangGraph**, utilizando estado, nós e conexões para organizar as etapas de execução do agente.
- Aplicar, de forma simples, conceitos de **arquitetura de agentes**, como separação entre planejamento, execução, uso de ferramentas e geração da resposta final.
- Integrar pelo menos uma **ferramenta** ao agente, como leitura ou escrita de arquivos, chamada a uma API, consulta a dados, análise de logs ou execução de uma função controlada.
- Utilizar **memória ou contexto durante a execução**, mantendo informações relevantes no estado do agente para apoiar o processamento e a resposta final.
- Aplicar cuidados básicos de **segurança e validação**, como controle das entradas recebidas, proteção de chaves de API, limitação de ações da ferramenta e geração de saídas verificáveis.
- Documentar o funcionamento do agente, os principais prompts utilizados, exemplos de entrada e saída e manter o projeto versionado no GitHub.

3. RESULTADOS ESPERADOS (ENTREGA)

O projeto deverá ser entregue até **20/07/2026 às 22h**, via **AVA**, por meio do envio do link do repositório GitHub. O mini-projeto corresponde a **30% da nota do módulo** e poderá ser desenvolvido **individualmente ou em grupos de até 3 alunos**.

O repositório deverá estar acessível e conter:

- **README.md** completo;
- código-fonte do agente implementado com **LangGraph**;
- pelo menos uma ferramenta integrada ao agente;
- exemplos de entrada e saída da execução;
- registro dos **principais prompts utilizados** em arquivo **.md**;
- apresentação da ideia do projeto em até **2 slides**.

A apresentação deverá ser submetida via AVA ou versionada no próprio repositório, conforme orientação do professor. Os slides devem apresentar, de forma objetiva, o

problema escolhido, o processo automatizado, a proposta do agente, as ferramentas utilizadas e o fluxo geral da solução.

Importante:

- Não serão aceitos projetos submetidos após a data limite.
- Teste o link do repositório antes da submissão para garantir que ele está acessível.
- Não modifique o repositório após a entrega até receber a nota.
- Não versione chaves de API, tokens ou informações sensíveis no repositório.
- Todos os alunos serão avaliados pelos mesmos critérios, sem divisão por nível de experiência.
- Em projetos em grupo, a entrega será coletiva, mas a avaliação será individual conforme a contribuição de cada integrante. Por isso, todos os alunos deverão manter evidências rastreáveis de participação no repositório, como commits, implementação, documentação, revisão ou organização da entrega.

4. REQUISITOS DA APLICAÇÃO

O projeto deverá atender aos seguintes requisitos técnicos e de execução:

1. Definir um processo real a ser automatizado, descrevendo o objetivo do agente, a entrada esperada, as etapas principais e a saída produzida.
2. Implementar o agente com **LangGraph**, utilizando um fluxo organizado com estado, nós e conexões entre as etapas.
3. Integrar pelo menos uma ferramenta ao agente, como leitura de arquivo, escrita de relatório, chamada a API, consulta a dados ou execução de função controlada.
4. Utilizar memória ou contexto durante a execução, mantendo informações relevantes no estado do agente ou em uma estrutura simples de apoio.
5. Registrar os principais prompts utilizados em arquivo **.md**, incluindo prompts usados para planejar, implementar, corrigir ou melhorar o agente.
6. Documentar no **README.md** como o agente funciona, como executar o projeto e quais decisões principais foram tomadas.
7. Manter o projeto versionado no GitHub. Em projetos em grupo, cada integrante deverá apresentar contribuição rastreável, pois a entrega será coletiva, mas a avaliação considerará a participação individual de cada aluno

5. ROTEIRO DA APLICAÇÃO

Abaixo seguem as etapas recomendadas para o desenvolvimento do mini-projeto.

5.1. FORMAÇÃO DOS GRUPOS E IDEIA INICIAL

Primeiros passos do projeto. Nesta etapa, o aluno ou grupo deverá definir o escopo da solução e apresentar a ideia inicial.

- Definir se o projeto será desenvolvido individualmente ou em grupo de até 3 alunos.
- Escolher a temática ou case do projeto.
- Definir qual processo será automatizado pelo agente.
- Criar o repositório no GitHub.
- Preparar uma apresentação de até 2 slides com a ideia do projeto.

A apresentação deve conter, de forma objetiva:

- problema escolhido;
- processo que será automatizado;
- proposta do agente;
- entrada esperada;
- saída esperada;
- visão geral do fluxo da solução.

5.2. DEFINIÇÃO DO AGENTE

O projeto deverá apresentar claramente o agente que será construído e qual papel ele terá dentro da solução.

- Definir o objetivo do agente.
- Informar quais entradas o agente irá receber.
- Informar quais saídas o agente deverá produzir.
- Descrever quais etapas principais o agente deverá executar.
- Explicar, de forma breve, por que a solução pode ser considerada um agente.

Exemplos de agentes possíveis:

- agente para revisar trechos de código;
- agente para analisar logs de pipeline;
- agente para classificar issues;
- agente para gerar relatórios técnicos;
- agente para consultar dados de uma API;
- agente para apoiar decisões dentro do case escolhido pelo aluno ou grupo.

5.3. IMPLEMENTAÇÃO COM LANGGRAPH, FERRAMENTA E CONTEXTO

O agente deverá ser implementado utilizando LangGraph, organizando o fluxo em etapas claras.

O projeto deve conter:

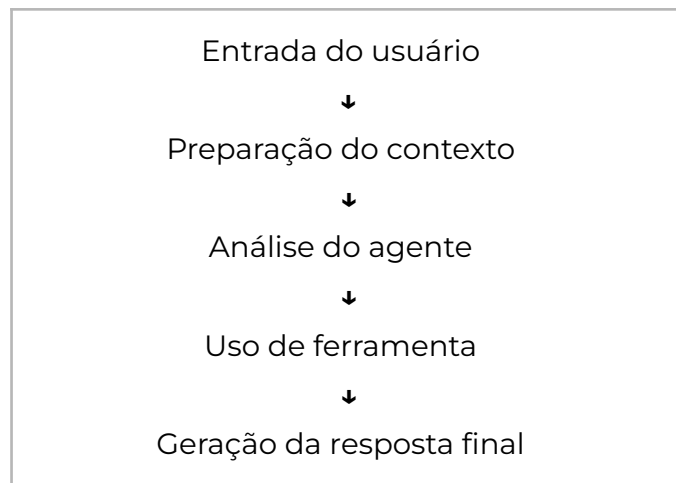
- estado compartilhado para armazenar informações da execução;
- nós responsáveis pelas etapas principais do processo;
- conexões entre os nós;
- pelo menos uma ferramenta integrada ao fluxo;
- uso de contexto ou memória durante a execução;
- geração de uma resposta final estruturada.

São exemplos de ferramentas aceitas:

- leitura de arquivo;
- escrita de relatório;
- chamada a uma API;
- consulta a dados locais;
- análise de logs;
- processamento de texto;
- execução de uma função controlada;
- busca de informações em documentos do projeto.

A ferramenta deve ter uma função clara dentro do processo automatizado, contribuindo para que o agente processe a entrada e gere uma saída útil.

Uma estrutura possível de fluxo seria:



O aluno ou grupo poderá adaptar esse fluxo conforme a necessidade do projeto.

5.4. DOCUMENTAÇÃO, PROMPTS E REPOSITÓRIO

O projeto deverá estar documentado de forma clara no repositório.

O **README.md** deve conter:

- nome do projeto;
- descrição do problema;
- objetivo do agente;
- explicação do fluxo com LangGraph;
- ferramenta utilizada pelo agente;
- instruções para executar o projeto;
- exemplo de entrada;
- exemplo de saída;
- principais decisões tomadas;
- limitações da solução.

Os principais prompts utilizados deverão ser registrados em um arquivo `.md`, por exemplo:

docs/prompts.md

Esse arquivo deve conter os prompts mais relevantes usados para planejar, implementar, corrigir ou melhorar o agente.

O repositório deve estar organizado, com histórico de commits compatível com o desenvolvimento realizado. Em projetos em grupo, cada integrante deverá apresentar contribuição rastreável, pois a entrega será coletiva, mas a avaliação considerará a participação individual de cada aluno.

6. CRITÉRIOS DE AVALIAÇÃO

A nota varia de **0 a 10 pontos** e corresponde a **30% da avaliação do módulo**. Todos os alunos serão avaliados pelos mesmos critérios, sem divisão por nível de experiência.

Projetos com plágio de soluções encontradas na internet ou de colegas receberão nota **0**. O uso de materiais, documentações e ferramentas de IA é permitido como apoio, desde que a solução entregue seja própria.

Nº	Critério de Avaliação	Pontuação
1	Versionamento com branches e commits semânticos	1,0
2	Contribuição individual e produtividade	1,0
3	Organização dos arquivos, documentação e prompts	2,0
4	Ideia do projeto e apresentação	1,0
5	Implementação do agente com LangGraph	1,0
6	Uso de ferramenta integrada ao agente	1,0

7	Cuidados básicos de segurança	1,0
8	Contexto, memória e validação básica	2,0
	Total	10,0

Uso do GitHub e colaboração

Nº	Critério de Avaliação	0	0,5	1,0
1	Versionamento com branches e commits semânticos	O repositório não apresenta branches e commits.	O repositório possui commits, mas as mensagens não seguem um padrão semântico consistente, dificultando a rastreabilidade das alterações.	O repositório possui commits claros, incrementais e alinhados ao padrão de commits semânticos, permitindo acompanhar a evolução do projeto.
2	Contribuição individual e produtividade	Sem evidências claras de participação.	Participação parcial ou pouco rastreável.	Contribuiu com a concepção da aplicação por meio de commits frequentes, implementação de funcionalidades, elaboração ou melhoria da documentação, revisão de código em Pull Requests ou organização da entrega coletiva.

Nº	Critério de Avaliação	0	1,0	2,0
3	Organização dos arquivos, documentação e prompts	O repositório não apresenta a documentação mínima solicitada ou possui arquivos desorganizados, dificultando a compreensão e execução do projeto.	O repositório apresenta parte da documentação solicitada, mas o README.md, os registros de prompts ou os exemplos de execução estão incompletos ou pouco organizados.	O repositório apresentou documentação completa e organizada, incluindo README.md, registro dos principais prompts, exemplos de entrada e saída e instruções claras de execução

Aplicação				
Nº	Critério de Avaliação	0	0,5	1,0
4	Ideia do projeto e apresentação	Não entregou slides ou a proposta não tem relação com agentes.	Elaborou uma apresentação com ideia, mas não evidenciou problema, agente ou fluxo.	Elaborou uma apresentação com até 2 slides, contendo problema, agente, entrada, saída e fluxo geral.
5	Implementação do agente com LangGraph	Não usa LangGraph ou não há agente funcional.	Utilizou LangGraph, mas o fluxo está incompleto, pouco claro ou com funcionamento parcial.	O código-fonte apresentou a implementação do agente por meio do framework LangGraph, utilizando um grafo de estados (StateGraph) para o controle de fluxo e tomada de decisão.
6	Uso de ferramenta integrada ao agente	Não utiliza ferramenta ou a ação é apenas simulada.	Usou ferramenta simples, mas com baixa integração ao fluxo.	O código-fonte apresentou a integração de ferramentas externas (como APIs, sistemas de arquivos ou funções customizadas) ao agente de IA, viabilizando a execução autônoma de ações estruturadas.
7	Cuidados básicos de segurança	O projeto expõe credenciais, tokens, chaves de API ou informações sensíveis no código ou no repositório.	O projeto demonstrou algum cuidado com informações sensíveis, mas ainda apresenta configuração pouco clara ou risco de exposição.	O projeto não expôs credenciais, tokens ou chaves de API, utiliza <code>.gitignore</code> para ignorar arquivos sensíveis e, quando necessário, fornece <code>.env.example</code> apenas com os nomes das variáveis, sem valores reais.

Nº	Critério de Avaliação	0	1,0	2,0
8	Contexto, memória e validação básica	Não demonstrou uso de contexto, memória ou validação mínima.	Usou contexto ou memória de forma limitada, com pouca validação de entrada, saída ou uso da ferramenta.	Usou estado ou memória para apoiar a execução do agente e implementou validações básicas de entrada, saída ou uso da ferramenta, evitando o processamento de dados malformados.

Regra da apresentação: a pontuação do **critério 1** será atribuída ao grupo, pois a apresentação representa a proposta coletiva. Um ou mais integrantes podem apresentar. Caso nenhum integrante apresente quando solicitado, o grupo não pontuará nesse critério.

Regra da avaliação individual: em projetos em grupo, a entrega é coletiva, mas a avaliação considera a contribuição individual de cada aluno. Por isso, integrantes do mesmo grupo podem receber notas diferentes no **critério 7**.

7. CHECKLIST FINAL DE ENTREGA

Antes de submeter no AVA, confira:

Repositório e organização

- ☐ Criei o repositório no GitHub e ele está acessível para avaliação.
- ☐ O repositório contém o código-fonte do agente.
- ☐ O projeto está organizado e possui histórico de commits compatível com o desenvolvimento realizado.
- ☐ Em projetos em grupo, cada integrante possui uma contribuição rastreável.

Agente e implementação

- ☐ Defini o processo que será automatizado pelo agente.
- ☐ O agente possui objetivo, entrada e saída claramente definidos.
- ☐ Implementei o agente usando **LangGraph**.
- ☐ O fluxo utiliza estado, nós e conexões entre etapas.
- ☐ O agente executa de forma funcional e gera uma saída estruturada.

Ferramentas, contexto e validação

- ☐ O agente utiliza pelo menos uma ferramenta integrada.
- ☐ A ferramenta executa uma ação real, como ler arquivo, escrever relatório, consultar dados, chamar API, analisar logs ou executar função controlada.
- ☐ O agente utiliza contexto ou memória durante a execução.
- ☐ A solução possui validação básica de entrada, saída ou uso da ferramenta.
- ☐ Não foram versionadas chaves, tokens ou informações sensíveis no repositório.

README.md e prompts

- ☐ O **README.md** apresenta o problema, o objetivo do agente e o funcionamento geral da solução.
- ☐ O **README.md** explica como executar o projeto.
- ☐ O **README.md** descreve o fluxo com LangGraph e a ferramenta utilizada.
- ☐ O **README.md** apresenta exemplo de entrada e saída.
- ☐ Registre os principais prompts utilizados em arquivo **.md**.

Apresentação

- ☐ Preparei a apresentação da ideia do projeto em até **2 slides**.
- ☐ Os slides apresentam o problema, a proposta do agente, a entrada, a saída, a ferramenta utilizada e o fluxo geral da solução.
- ☐ A apresentação foi submetida via AVA ou versionada no repositório, conforme orientação do professor.

Submissão

- ☐ Submeti o link do repositório GitHub no AVA.
- ☐ Conferi se o link está acessível antes da submissão.
- ☐ Realizei a entrega antes do prazo: **20/07/2026 às 15h**.
- ☐ Não modificarei o repositório após a entrega até receber a nota.